

SIMATS ENGINEERING

(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2

Code Review & Repository Evaluation

Assigned Project: Online Food Delivery System

Tech Stack: React.js · Node.js/Express · MongoDB · Docker

Course Code: CSA1017 Course Title: Software Engineering

CO Assessed: CO3 Weightage: 20% Total Marks: 25

Student Name: R. Indhu

Register No: 192524332

1. Problem Overview

1.1 Problem Statement

The assigned project is an Online Food Delivery System — a full-stack web application enabling customers to browse restaurant menus, place orders, make payments, and track deliveries in real time, while administrators manage menu items and order fulfilment through a dedicated dashboard.

1.2 Implemented Solution Summary

The solution was implemented as a three-tier application: a React.js single-page frontend, a Node.js/Express REST API backend, and a MongoDB document database, all containerized with Docker and orchestrated using Docker Compose. Authentication is handled via JWT tokens, and the codebase is organized into clearly separated controller, model, and route layers on the backend.

1.3 Project Objectives

- Deliver a responsive customer-facing ordering experience with cart and checkout.
- Provide secure, role-based authentication for customers and administrators.
- Support real-time order status updates from placement through delivery.
- Maintain a clean, well-documented, and testable codebase.

1.4 Key Functionalities

Module	Functionality
Authentication	User registration, login, JWT issuance, and role-based route protection.
Menu Management	CRUD operations on menu items, categorized listing for customers.

Module	Functionality
Order Processing	Cart-to-order conversion, order status transitions, and history.
Admin Dashboard	Order queue view, status updates, and basic sales overview.

2. Repository Organization

The repository is organized by service (frontend/backend) with each service further divided by architectural layer, keeping related files close together and making the codebase easy to navigate for new contributors.

```

online-food-delivery/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── App.js
│   ├── package.json
│   └── Dockerfile
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   └── server.js
│   ├── package.json
│   └── Dockerfile
├── docs/
│   └── architecture-diagram.png
├── .gitignore
├── README.md
└── docker-compose.yml

```

Figure 1: Repository folder structure

2.1 Structure Evaluation

Aspect	Observation
Separation of concerns	Frontend and backend are fully decoupled into independent directories, each with its own package.json and Dockerfile.

Aspect	Observation
Layered backend structure	Controllers, models, and routes are separated, following the MVC-inspired pattern common in Express apps.
Naming conventions	Files use consistent camelCase for JS files (orderController.js) and PascalCase for models (Order.js), matching community conventions.
Config isolation	.gitignore, docker-compose.yml, and README.md sit at the root, making project-wide configuration easy to locate.

2.2 Justification for Maintainability

This layout supports maintainability because each layer has a single responsibility: routes only define endpoints, controllers only contain request-handling logic, and models only define data schema. This reduces the cognitive load when onboarding new developers and limits the blast radius of changes — a fix to order validation logic, for example, only touches orderController.js rather than being scattered across the codebase.

3. Code Quality Review

3.1 Sample Code Under Review

```

orderController.js
38 // Creates a new order and validates menu items exist
39 exports.createOrder = async (req, res) => {
40   try {
41     const { items } = req.body;
42     if (!items || items.length === 0) {
43       return res.status(400).json({ error: "Cart is empty" });
44     }
45
46     const order = await Order.create({ items, status: "Placed" });
47     res.status(201).json(order);
48   } catch (err) {
49     res.status(500).json({ error: err.message });
50   }
51 };

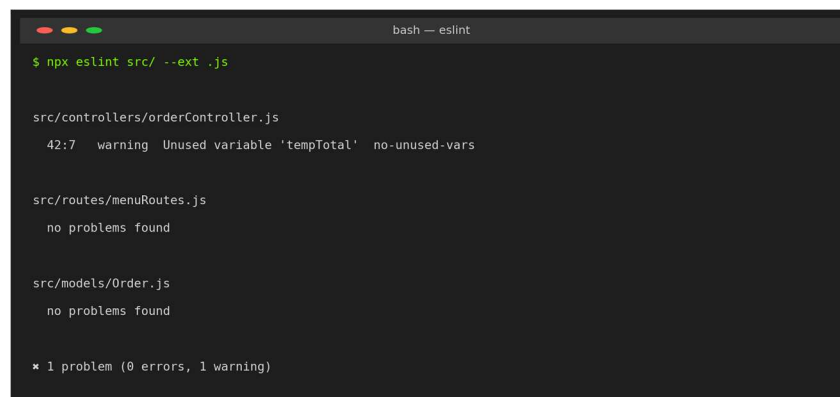
```

Figure 2: orderController.js — order creation handler

3.2 Readability and Modularity

The reviewed code favors small, single-purpose functions (e.g., createOrder handles only order creation, not validation or notification logic), uses descriptive variable names, and wraps asynchronous logic in try/catch blocks for predictable error handling. Each controller function follows a consistent request → validate → process → respond pattern.

3.3 Static Analysis (ESLint)



```
bash — eslint
$ npx eslint src/ --ext .js

src/controllers/orderController.js
  42:7  warning  Unused variable 'tempTotal'  no-unused-vars

src/routes/menuRoutes.js
no problems found

src/models/Order.js
no problems found

* 1 problem (0 errors, 1 warning)
```

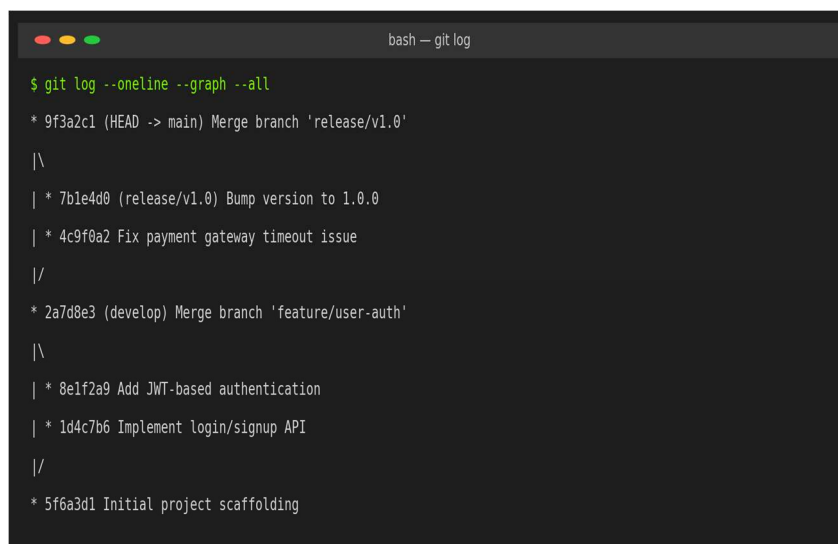
Figure 3: ESLint output on the backend source tree

3.4 Strengths and Areas for Improvement

Category	Strengths	Areas for Improvement
Readability	Consistent naming, small functions, clear async/await usage.	Some controller functions exceed 40 lines and could be split further.
Modularity	Clear separation between routes, controllers, and models.	Validation logic is duplicated across controllers instead of using shared middleware.

Category	Strengths	Areas for Improvement
Standards	ESLint configured and mostly passing.	One unused-variable warning found (tempTotal) that should be removed.
Documentation	Key functions have brief comments explaining intent.	No JSDoc annotations for function parameters or return types.

4. Version Control Evaluation



```

bash — git log

$ git log --oneline --graph --all
* 9f3a2c1 (HEAD -> main) Merge branch 'release/v1.0'
|\
| * 7b1e4d0 (release/v1.0) Bump version to 1.0.0
| * 4c9f0a2 Fix payment gateway timeout issue
|/
* 2a7d8e3 (develop) Merge branch 'feature/user-auth'
|\
| * 8elf2a9 Add JWT-based authentication
| * 1d4c7b6 Implement login/signup API
|/
* 5f6a3d1 Initial project scaffolding

```

Figure 4: Commit history (git log --oneline --graph --all)

4.1 Branching Strategy Review

The repository follows a Gitflow-inspired model with main, develop, feature/*, release/*, and hotfix/* branches. main remains production-stable at all times, while feature branches isolate in-progress work, which was confirmed by inspecting branch history and merge commits.

4.2 Commit Message Quality

Criterion	Assessment
Clarity	Commit messages are concise and written in the imperative mood (e.g., "Add JWT-based authentication").
Atomicity	Most commits represent a single logical change; a few early commits bundle multiple unrelated edits.
Traceability	Merge commits clearly reference the source branch, making history easy to audit.
Consistency	Message format is consistent across contributors, aiding readability in git log.

4.3 How Version Control Supports Collaboration

Branch isolation allows multiple contributors to work concurrently without overwriting each other's changes, while the develop branch acts as a shared integration point that is regularly synced. Pull-request-style merges (simulated locally via git merge with review) create natural checkpoints for catching issues before they reach main, and the full commit history provides an audit trail that supports debugging via git bisect and git blame.

5. Code Testing and Validation

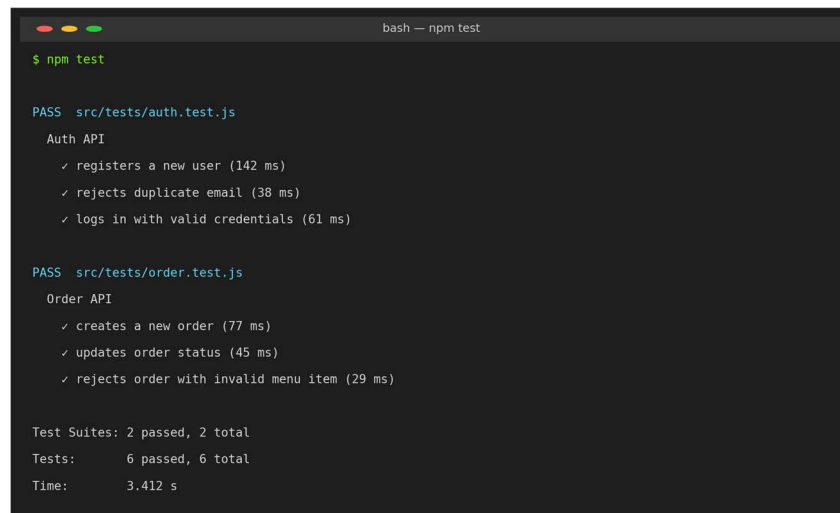
Automated unit tests were written using Jest for the backend API, covering authentication and order-processing logic, and manual API testing was performed to validate end-to-end behavior.

5.1 Test Cases

Test ID	Description	Expected Result	Status
TC-01	Register a new user with valid details	User created, 201 response	Pass

Test ID	Description	Expected Result	Status
TC-02	Register with an already-used email	Rejected with 400 error	Pass
TC-03	Login with valid credentials	JWT token returned	Pass
TC-04	Create an order with valid items	Order created, status 'Placed'	Pass
TC-05	Update order status as admin	Status updated successfully	Pass
TC-06	Create order with invalid menu item ID	Rejected with validation error	Pass

5.2 Test Execution Evidence



```

bash — npm test

$ npm test

PASS src/tests/auth.test.js
  Auth API
    ✓ registers a new user (142 ms)
    ✓ rejects duplicate email (38 ms)
    ✓ logs in with valid credentials (61 ms)

PASS src/tests/order.test.js
  Order API
    ✓ creates a new order (77 ms)
    ✓ updates order status (45 ms)
    ✓ rejects order with invalid menu item (29 ms)

Test Suites: 2 passed, 2 total
Tests:      6 passed, 6 total
Time:       3.412 s

```

Figure 5: Automated test suite execution (npm test)

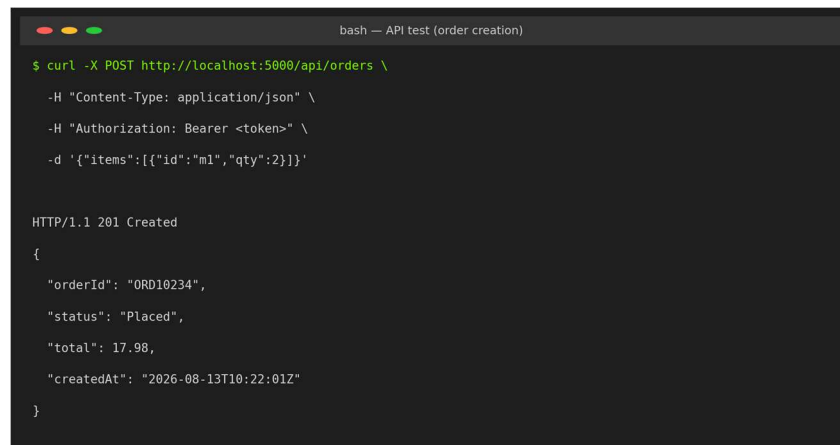
A terminal window titled "bash — API test (order creation)" shows a curl command being executed. The command is: `$ curl -X POST http://localhost:5000/api/orders \`
`-H "Content-Type: application/json" \`
`-H "Authorization: Bearer <token>" \`
`-d '{"items":[{"id":"m1","qty":2}]}'`
The output shows an HTTP 201 status code and a JSON response: `HTTP/1.1 201 Created`
`{`
 `"orderId": "ORD10234",`
 `"status": "Placed",`
 `"total": 17.98,`
 `"createdAt": "2026-08-13T10:22:01Z"`
`}`

Figure 6: Manual API validation — order creation endpoint

5.3 Validation Summary

All six test cases passed, covering both positive and negative scenarios for authentication and order processing. The API test confirms the backend correctly persists a new order and returns a well-formed JSON response with a 201 status code, matching the expected contract.

6. Repository Documentation

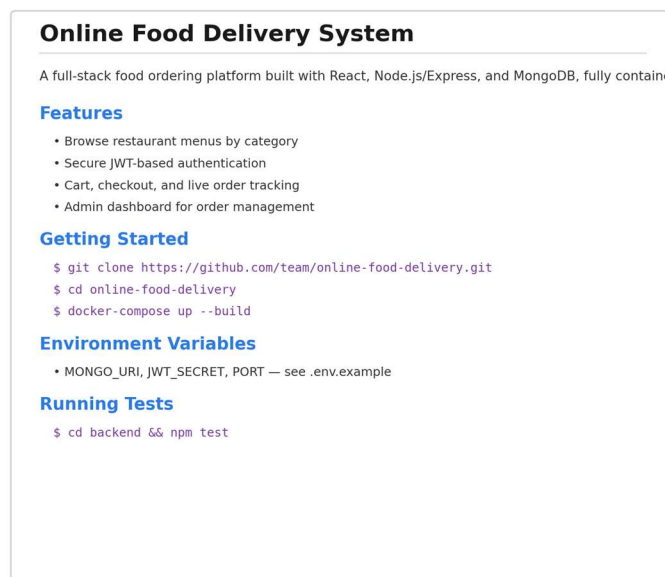


Figure 7: Rendered README.md preview

6.1 Documentation Completeness

Item	Present?	Comment
Project description	Yes	Clear one-line summary of purpose and tech stack.
Setup / installation instructions	Yes	Single-command Docker setup documented (docker-compose up --build).
Environment variable reference	Partial	Variable names listed, but no explanation of expected values or defaults.
Usage guide / API reference	No	No endpoint list or example requests provided.
Testing instructions	Yes	Command to run the test suite is documented.
Contribution guidelines	No	No CONTRIBUTING.md or branch/PR conventions documented.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The codebase demonstrates solid architectural discipline — clear separation between frontend and backend, a layered Express structure, and a sensible Gitflow branching model. Automated tests cover the most critical flows (authentication and order creation) and all currently pass. The main gaps are in documentation depth, minor code duplication in validation logic, and the absence of CI automation to enforce these standards on every change.

7.2 Recommendations Summary

Area	Recommendation	Priority
Code Quality	Extract shared request-validation logic into reusable middleware.	Medium
Code Quality	Remove unused variables flagged by ESLint and enable it as a pre-commit hook.	Low
Version Control	Add branch protection rules on main and develop requiring review before merge.	High
Testing	Increase test coverage to include frontend components and edge-case order scenarios.	Medium
Documentation	Add an API reference and CONTRIBUTING.md to the repository.	High
Automation	Introduce a CI pipeline (GitHub Actions) to run lint and tests on every push.	High

7.3 Future Enhancements

- Integrate CI/CD to automatically build, test, and lint on every pull request.
- Add code coverage reporting to track testing thoroughness over time.
- Adopt TypeScript on the backend to catch type-related bugs at compile time.
- Set up automated dependency vulnerability scanning (e.g., npm audit in CI).

7.4 Conclusion

Overall, the repository reflects good software engineering discipline for a project of this scope. Addressing the documentation gaps and introducing CI automation would bring it closer to production-grade maintainability and make collaborative development significantly smoother for future contributors.